TypeScript Module 2: হ্যান্ড নোট

Union, Interface, Types, Assertion, Narrowing, Generics, Promises

2-1 TS মডিউল সেটআপ

- প্রতিটা `.ts` ফাইল একটা **মডিউল** হিসেবে কাজ করে, যদি তাতে `export` বা `import` থাকে।
- `export` করলে ফাইলের ভেতরের নাম গ্লোবাল স্কোপে মিশে যায় না, ফলে এক ফাইলের নাম আরেক ফাইলের সাথে ক্ল্যাশ করে না।

```
export const greeting = () => {
  console.log("hello arman");
};
// অন্য ফাইলে:
// import { greeting } from "./v1-module-config";
```

2-2 Type Union

ধরন	মানে	উদাহরণ
Literal type	একটাই নির্দিষ্ট মান	`type Status = 'pending'`
Basic union	একাধিক টাইপের যেকোনো একটা	`string | number`
Literal union	কয়েকটা নির্দিষ্ট মানের মধ্যে একটা, বাইরের মান দেওয়া যাবে না	`"user" | "admin" | "manager"`
Object union	একাধিক অবজেক্ট শেপের যেকোনো একটা	`Admin | Member`

```
type Role = "user" | "admin" | "manager";
const userRole: Role = "admin";      // ঠিক
// const bad: Role = "guest";        // error

type Admin  = { id: number; role: "admin";  privileges: string[] };
type Member = { id: number; role: "member"; plans: string[] };
type User = Admin | Member;
const user: User = { id: 125, role: "member", plans: ["ads"] };
```

2-3 Type alias বনাম Interface

	type alias	interface
কীসের জন্য	literal, union, tuple, primitive	মূলত অবজেক্টের শেপ
primitive (`number`) বানানো	✅ পারে	❌ পারে না
union বানানো	✅ পারে	❌ নিজে পারে না
বাড়ানো (extend)	`&` (intersection)	`extends`
একই নামে redeclare	❌ error	✅ merge হয়ে যায়

```
// interface extend
interface ID { id: number }
interface Information extends ID { name: string }

// type extend (intersection)
type Vehicle = { brand: string };
type Car = Vehicle & { model: string };

// declaration merging (শুধু interface-এ)
interface IEmployee { name: string }
interface IEmployee { id: number }   // দুটো মিলে একটা হয়ে যায়
const emp: IEmployee = { name: "arman", id: 1354 };
```

> **মনে রাখো:** একই নামে দুটো `type` লিখলে error আসে, কিন্তু দুটো `interface` লিখলে TypeScript নিজেই merge করে নেয়।

2-4 Type Hierarchy (Structural Typing)

- TypeScript নাম দেখে না, **শেপ** দেখে। যে টাইপে বেশি প্রপার্টি আছে, সেটা কম প্রপার্টির টাইপে অ্যাসাইন করা যায়।
- উল্টোটা করা যায় না, কারণ কম প্রপার্টির ভ্যালুতে বাকি প্রপার্টি থাকে না।

```
type Person  = { name: string };
type Student = { name: string; roll: number };

const myStudent: Student = { name: "arman", roll: 135 };
const myAll: Person = myStudent;   // ঠিক: Student-এ name আছেই
// const s: Student = myAll;       // error: roll নেই
```

2-5 বিভিন্ন ধরনের Special Type

ক্যাটাগরি	টাইপ	কাজ
Top type	`any`	যেকোনো কিছু, টাইপ চেক বন্ধ। ঝুঁকিপূর্ণ।
Top type	`unknown`	যেকোনো কিছু রাখা যায়, কিন্তু ব্যবহার করার আগে চেক বা assertion লাগে। বেশি নিরাপদ।
Nullish	`null`, `undefined`	মান নেই বোঝাতে, যেমন `User | null`
Unit	`void`	কিছু রিটার্ন করে না এমন ফাংশন
Unit-এর subtype	`undefined`	স্পষ্টভাবে `undefined` রিটার্ন
Bottom type	`never`	যে ফাংশন কখনো রিটার্ন করে না (error throw করে)

```
let x: any = "hello"; x = 451;          // কোনো বাধা নেই

let something: unknown = 451;
// something.length                     // error: আগে চেক করতে হবে
const some = something as number;       // assertion

type LoggedInUser = User | null;
const currentUser: LoggedInUser = null;

const getError = (msg: string): never => {
  throw new Error(msg);
};
```

2-6 Type Assertion

> **সতর্কতা:** assertion আসলে কম্পাইলারকে বলা যে "আমি জানি, তুমি মেনে নাও"। যত কম ব্যবহার করা যায় ততই ভালো। ব্যবহার করার আগে নিজেকে জিজ্ঞেস করো, কেন লাগছে।

```
// 1) "as" syntax
type Status = "Pending" | "Approved" | "Rejected";
const someStatus: string = "Pending";
showStatus(someStatus as Status);

// 2) non-null assertion  ( ! )
interface Config { port?: number }
const config: Config = { port: 5000 };
const p1: number = config.port!;          // "null/undefined নয়" বলা
const p2: number = config.port as number; // একই কাজ

// 3) constant assertion
const configuration = { port: 8080, prefix: "api/v2" } as const;
// সব প্রপার্টি readonly + literal টাইপ হয়ে যায়
```

2-7 Narrowing (পর্ব ১)

Narrowing মানে বিস্তৃত টাইপ (যেমন `string | number`) থেকে চেক করে নির্দিষ্ট টাইপে নামিয়ে আনা।

```
// typeof type guard
const print = (value: string | number) => {
  if (typeof value === "string") return value.toUpperCase(); // এখানে string
  return value;                                              // এখানে number
};

// Truthiness narrowing
type User = { name: string; address: { street: string } } | null;
const showAddress = (user: User) => {
  if (user && user.address) {
    console.log(user.address.street);
    return;
  }
  throw new Error("user not found");
};

// বিকল্প: optional chaining
// console.log(user?.address.street);
```

2-8 Narrowing (পর্ব ২)

```
// Array.isArray guard
const handleSearch = (filters: string | string[] | null) => {
  if (Array.isArray(filters)) {
    for (const s of filters) console.log(s);
  } else if (typeof filters === "string") {
    console.log(filters);
  } else {
    console.log("Showing all products");
  }
};

// Equality narrowing
const printAll = (x: string | string[] | null) => {
  if (x !== null) {            // আগে null বাদ
    if (typeof x === "object") {
      for (const s of x) console.log(s);
      return;
    }
    console.log(x);
  }
};

// "in" operator
type Bird = { fly: () => void };
type Fish = { swim: () => void };
const move = (a: Bird | Fish) => {
  if ("fly" in a) { a.fly(); return; }
  a.swim();
};

// instanceof
const showDate = (x: Date | string) => {
  if (x instanceof Date) console.log(x.toISOString());
};

// Discriminated union
interface Circle { kind: "circle"; radius: number }
interface Square { kind: "square"; sideLength: number }
const getArea = (s: Circle | Square) =>
  s.kind === "circle" ? Math.PI * s.radius ** 2 : s.sideLength ** 2;
```

JS ট্র্যাপ: `typeof null === "object"`। তাই `typeof x === "object"` চেকের আগে `x !== null` চেক করা জরুরি।

গার্ড	কখন ব্যবহার করবে
`typeof`	primitive (string, number, boolean)
truthiness / `!== null`	null/undefined বাদ দিতে
`Array.isArray()`	অ্যারে চেনার জন্য
`in`	প্রপার্টি আছে কিনা দেখে অবজেক্ট আলাদা করতে
`instanceof`	ক্লাস/বিল্ট-ইন অবজেক্ট (Date, Error)
`kind` প্রপার্টি	Discriminated union, সবচেয়ে পরিষ্কার

2-9 Generics

- Generic মানে "টাইপ পরে ঠিক হবে", `<T>` একটা টাইপ-প্লেসহোল্ডার।
- একই কোড বিভিন্ন টাইপের জন্য আবার না লিখে কাজ চালানো যায়, আবার টাইপ-সেফটিও থাকে।

```
const wrapInArray = <T>(value: T) => [value];
const stringArr = wrapInArray<string>("arman");   // string[]
const numberArr = wrapInArray<number>(127705);    // number[]
stringArr.map((item) => item.toUpperCase());      // item হলো string

interface ApiResponse<T> {
  data: T;
  success: boolean;
  statusCode: number;
}
type UserRes = ApiResponse<User>;
type BlogRes = ApiResponse<Blog>;
```

2-10 Promises ও Async

- `async` ফাংশন সবসময় `Promise` রিটার্ন করে, তাই রিটার্ন টাইপ হবে `Promise<T>`।
- `fetch` এর `res.json()` টাইপ দেয় না, তাই `as ApiResponse<User>` দিয়ে বলে দিতে হয়।
- `catch` এর `error` কী টাইপের তা নিশ্চিত নয়, তাই `instanceof Error` দিয়ে narrow করতে হয়।

```
const fetchUser = async (): Promise<User | undefined> => {
  try {
    const res = await fetch("/api/v1/some-endpoint");
    const data = (await res.json()) as ApiResponse<User>;
    if (!res.ok || !data.success) {
      throw new Error(`Failed fetch user data: ${data.message}`);
    }
    return data.data;
  } catch (error) {
    if (error instanceof Error) console.error(error.message);
    else console.error("something went wrong");
  }
};
```

! সাধারণ ভুল (তোমার ফাইল থেকে)

ফাইল	ভুল	ঠিক
v8 `showDate`	`Date | String`	`Date | string` (ছোট হাতের)
v8 `printAll`	`for` লুপের ভেতরে `return`, প্রথম আইটেমের পর থেমে যায়	লুপের বাইরে `return`
v10	`catch` এ ফেরত না দিলে `undefined` যায়	রিটার্ন টাইপে `| undefined` রাখা ঠিকই আছে

★ এক নজরে চিটশিট

দরকার	ব্যবহার করো
নির্দিষ্ট কয়েকটা মান	Literal union
অবজেক্টের শেপ, পরে বাড়ানো হতে পারে	`interface`
union, tuple, primitive alias	`type`
টাইপ জানা নেই, নিরাপদে ধরতে চাও	`unknown` + narrowing
কখনো রিটার্ন করে না	`never`
একই লজিক বিভিন্ন টাইপে	Generic `<T>`
API রেসপন্সের শেপ	`ApiResponse<T>`
async ফাংশনের রিটার্ন	`Promise<T>`